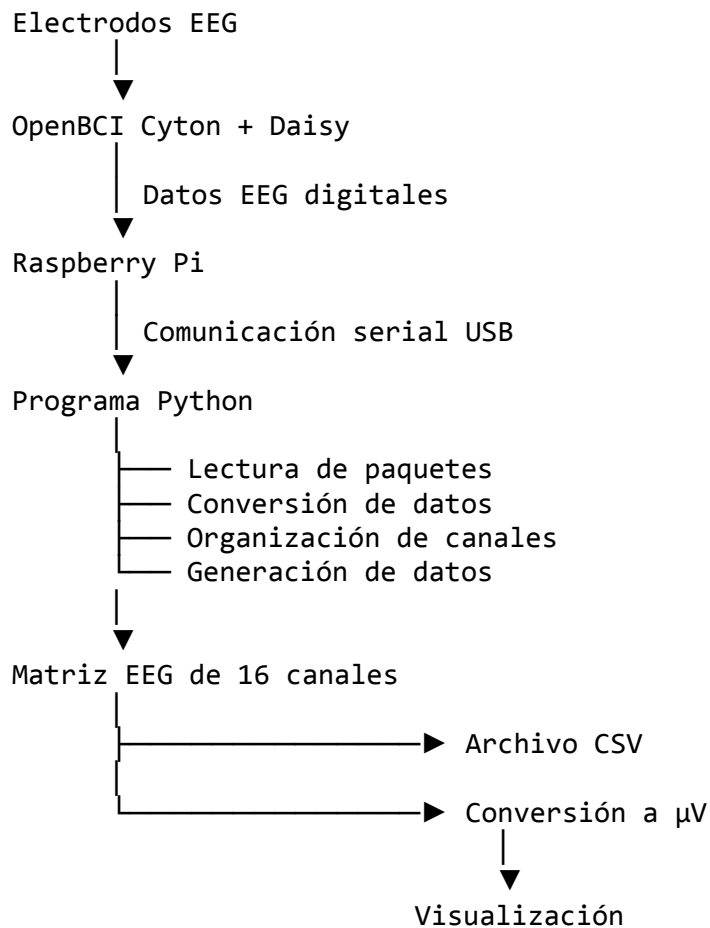


Arquitectura del sistema

1. Descripción general

El sistema desarrollado permite realizar la adquisición de señales EEG utilizando un sistema **OpenBCI Cyton + Daisy de 16 canales**, conectado a una **Raspberry Pi** encargada de ejecutar el programa de adquisición desarrollado en Python.

La arquitectura está organizada en varias etapas:



El flujo general puede dividirse en cuatro bloques principales:

1. **Adquisición de la señal**
 2. **Transmisión de los datos**
 3. **Procesamiento y organización**
 4. **Almacenamiento y visualización**
-

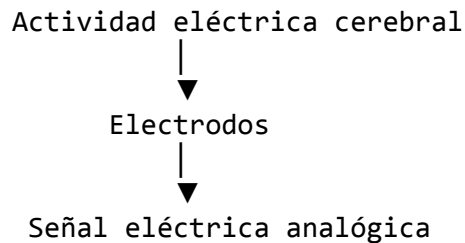
2. Componentes del sistema

2.1 Electrodo

Los electrodos son el punto de entrada de la se al biol gica.

Su funci n es detectar las variaciones de potencial el ctrico producidas por la actividad cerebral y proporcionar estas se ales al sistema de adquisici n.

En este proyecto, los electrodos se conectan a las entradas correspondientes del sistema OpenBCI.



Los electrodos no realizan el procesamiento principal de los datos. La adquisici n, amplificaci n y digitalizaci n son realizadas posteriormente por el hardware OpenBCI.

3. OpenBCI Cyton + Daisy

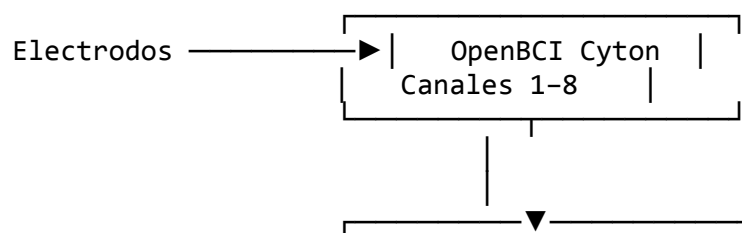
El sistema de adquisici n est  compuesto por una placa **OpenBCI Cyton** y una expansi n **Daisy**.

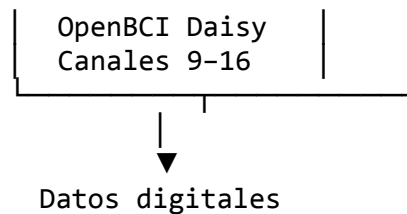
La Cyton proporciona 8 canales de adquisici n, mientras que la expansi n Daisy permite trabajar con un total de 16 canales.

La funci n principal de esta etapa es:

- Recibir las se ales provenientes de los electrodos.
- Acondicionar las se ales.
- Digitalizar las se ales.
- Generar los paquetes de datos.
- Transmitir los datos hacia el equipo de procesamiento.

Conceptualmente:





Cada paquete individual contiene información correspondiente a **8 canales EEG**.

Por esta razón, el programa de Python utiliza dos paquetes para construir una muestra de trabajo con los 16 canales.

4. Raspberry Pi

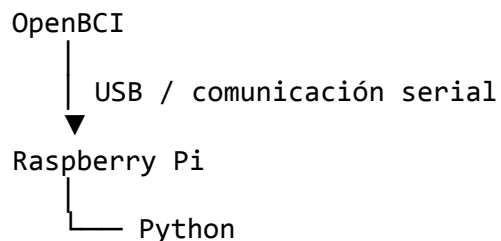
La Raspberry Pi funciona como la unidad de procesamiento del sistema.

En ella se ejecuta el programa desarrollado en Python, encargado de recibir y organizar los datos enviados por el sistema OpenBCI.

Sus principales responsabilidades son:

- Establecer la comunicación serial.
- Inicializar el dispositivo.
- Solicitar el inicio del streaming.
- Recibir los paquetes de datos.
- Validar los paquetes recibidos.
- Extraer los valores de los canales EEG.
- Construir la matriz de datos.
- Guardar los resultados.
- Generar la visualización de las señales.

La arquitectura queda entonces:



5. Comunicación entre OpenBCI y Raspberry Pi

La comunicación se realiza mediante el puerto serial disponible en la Raspberry Pi.

En el programa se establece la conexión mediante:

```
ser = serial.Serial('/dev/ttyUSB0', 115200, timeout=1)
```

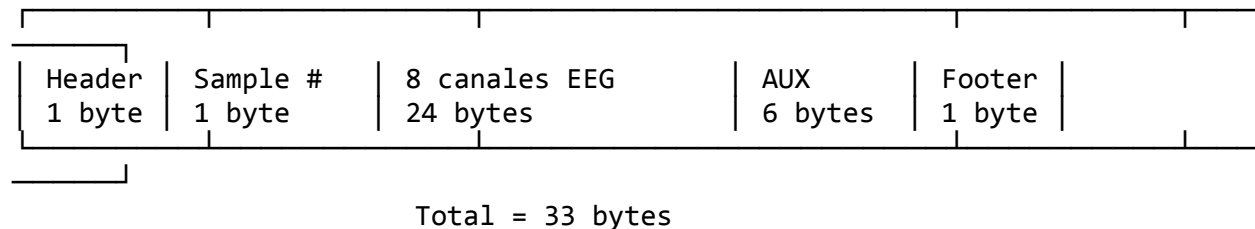
Los parámetros principales son:

Parámetro	Valor
Puerto	/dev/ttyUSB0
Velocidad	115200 baudios
Timeout	1 segundo

El puerto /dev/ttyUSB0 corresponde al dispositivo serial detectado por el sistema operativo Linux.

La información recibida por Python se encuentra organizada en paquetes de **33 bytes**.

Cada paquete contiene:



La descripción detallada de esta estructura se encuentra en:

[docs/comunicacion_serial.md](#)

6. Inicialización del sistema

Antes de comenzar la adquisición, el programa realiza una secuencia de inicialización.

6.1 Conexión serial

Primero se abre el puerto serial:

```
ser = serial.Serial('/dev/ttyUSB0', 115200, timeout=1)
```

Posteriormente se espera un breve periodo para permitir que la conexión se establezca correctamente:

```
time.sleep(2)
```

6.2 Reinicio del dispositivo

El programa envía:

```
ser.write(b'v')
```

Este comando se utiliza para reinicializar el dispositivo.

6.3 Configuración de canales

Después se envía:

```
ser.write(b'd')
```

Con esto se establece la configuración utilizada por el proyecto para la adquisición.

6.4 Inicio del streaming

Antes de comenzar la lectura se limpia el buffer:

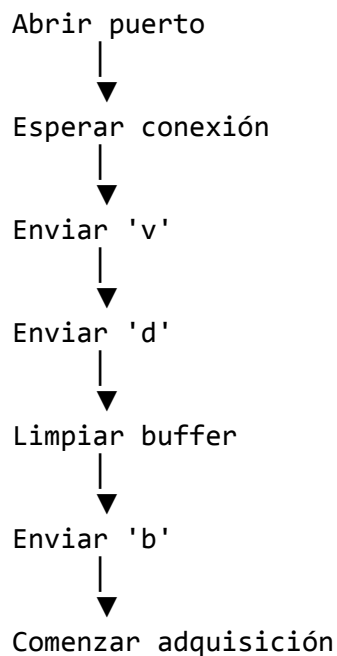
```
ser.reset_input_buffer()
```

Posteriormente se envía:

```
ser.write(b'b')
```

para iniciar el streaming de datos.

El flujo puede representarse como:



7. Adquisición de los 16 canales

Una característica importante de la implementación es que cada paquete contiene información de 8 canales.

Por lo tanto, el programa lee dos paquetes:

```
packet1 = ser.read(33)
packet2 = ser.read(33)
```

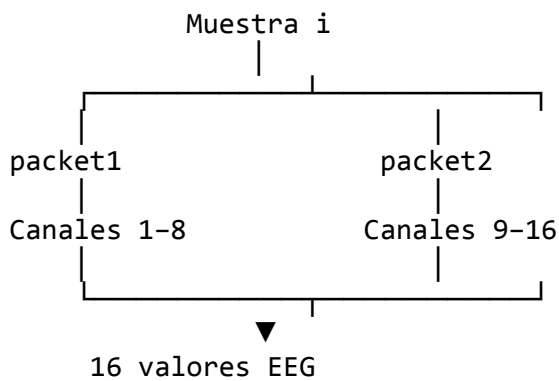
El primer paquete se utiliza para obtener los canales 1–8:

```
packet1
├── Canal 1
├── Canal 2
├── Canal 3
├── Canal 4
├── Canal 5
├── Canal 6
├── Canal 7
└── Canal 8
```

El segundo paquete se utiliza para obtener los canales 9–16:

```
packet2
├── Canal 9
├── Canal 10
├── Canal 11
├── Canal 12
├── Canal 13
├── Canal 14
├── Canal 15
└── Canal 16
```

El programa combina ambos grupos dentro de una misma estructura de datos:



Esta organización permite trabajar posteriormente con una matriz de 16 canales.

8. Estructura de almacenamiento en memoria

La matriz principal se crea mediante:

```
data_EEG = np.zeros((16, N_points))
```

En la implementación actual:

```
N_points = 1000
```

Por lo tanto, la matriz tiene dimensiones:

16 × 1000

donde:

- 16 = número de canales.
- 1000 = número de muestras almacenadas por canal.

Visualmente:



Cada fila representa un canal y cada columna representa una posición temporal dentro de la adquisición.

9. Extracción de los datos

Los datos de cada canal ocupan tres bytes dentro del paquete.

El programa utiliza:

```
idx = 2 + ch*3
```

para localizar los tres bytes correspondientes a cada canal.

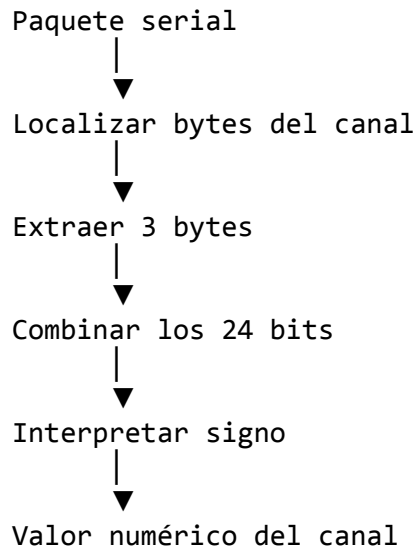
Posteriormente se convierten mediante:

```
read_24bit_signed(  
    packet[idx],  
    packet[idx+1],
```

```
) packet[idx+2]
```

El resultado es un valor entero con signo de 24 bits.

El procedimiento completo es:



El funcionamiento detallado de esta conversión se documenta en docs/comunicacion_serial.md y docs/procesamiento.md.

10. Almacenamiento de los resultados

Una vez finalizada la adquisición, el programa detiene el streaming:

```
ser.write(b's')  
ser.close()
```

Posteriormente, la matriz se convierte en un DataFrame de Pandas.

Debido a que la matriz original está organizada como:

16 canales × 1000 muestras

se utiliza:

data_EEG.T

para obtener:

1000 muestras × 16 canales

El archivo CSV queda estructurado de la siguiente manera:

Canal_1	Canal_2	Canal_3	...	Canal_16
x1	x1	x1	...	x1
x2	x2	x2	...	x2
x3	x3	x3	...	x3
...	...	...	...	...

El archivo generado es:

EEG_16_canales.csv

11. Visualización

Además del almacenamiento de los datos, el sistema genera una representación gráfica de las señales.

Primero se genera un eje temporal:

```
t = np.arange(N_points) / fs
```

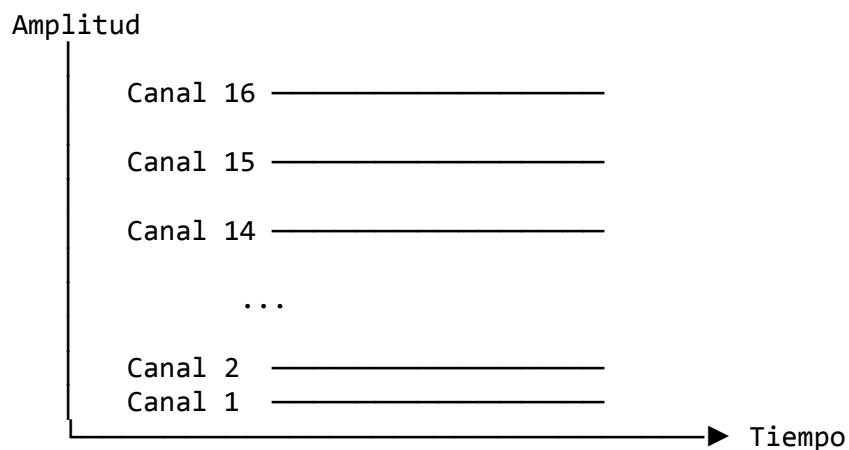
Después se convierten los valores utilizando el factor definido en el programa para expresarlos en microvoltios:

```
data_EEG_uV = data_EEG * scale
```

Finalmente, las señales de los 16 canales se muestran utilizando diferentes desplazamientos verticales:

```
plt.plot(t, data_EEG_uV[ch] + ch*offset)
```

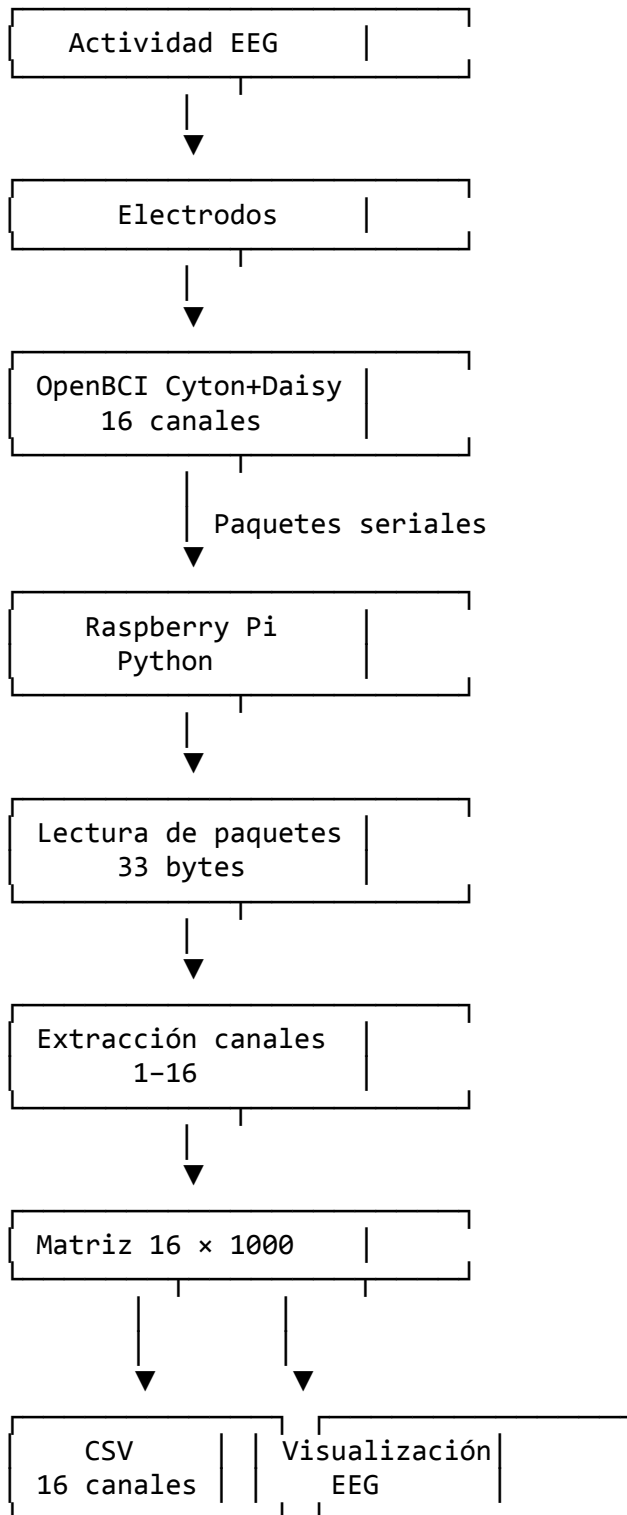
El resultado conceptual es:



El desplazamiento vertical permite visualizar simultáneamente los 16 canales sin que las señales queden superpuestas.

12. Flujo completo del sistema

El funcionamiento completo puede resumirse de la siguiente manera:



13. Responsabilidades de cada componente

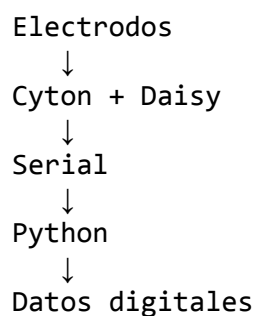
Componente	Responsabilidad
Electrodos	Captar la actividad eléctrica
Cyton	Adquisición y digitalización de canales
Daisy	Ampliar el sistema hasta 16 canales
Raspberry Pi	Ejecutar el sistema de adquisición
Puerto serial	Transportar los datos hacia Python
Python	Recibir y procesar los paquetes
NumPy	Administrar la matriz de datos
Pandas	Generar el archivo CSV
Matplotlib	Visualizar las señales
CSV	Almacenar los datos adquiridos

14. Separación entre adquisición y procesamiento

Una característica importante de la arquitectura es la separación conceptual entre **adquisición** y **procesamiento**.

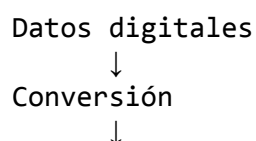
Adquisición

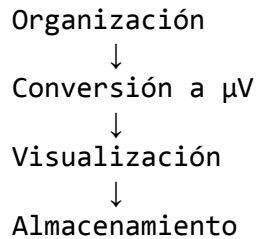
La etapa de adquisición se encarga de obtener los datos desde el hardware:



Procesamiento

La etapa posterior utiliza los datos adquiridos para:





Esta separación facilita futuras ampliaciones del proyecto, ya que se pueden agregar algoritmos de procesamiento sin modificar necesariamente la parte encargada de comunicarse con el hardware.

15. Posibles ampliaciones

La arquitectura actual puede utilizarse como base para agregar posteriormente:

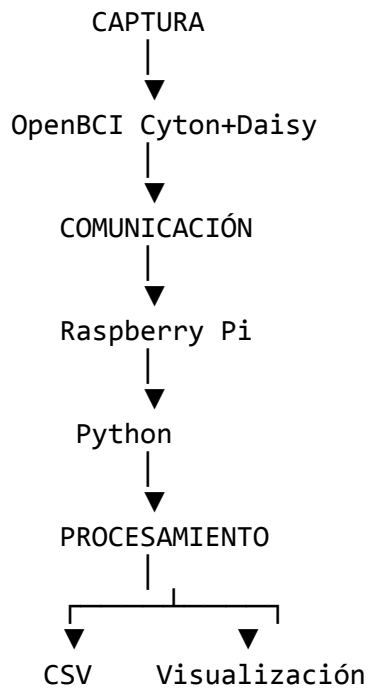
- Filtros digitales.
- Filtro notch para interferencia de red eléctrica.
- Filtros pasa-bajas y pasa-altas.
- Eliminación o detección de artefactos.
- FFT.
- Análisis de potencia espectral.
- Espectrogramas.
- Análisis de bandas EEG.
- Detección de eventos.
- Clasificación de señales.
- Visualización en tiempo real.
- Base de datos para almacenar sesiones.
- Interfaz gráfica para controlar la adquisición.
- Transmisión de datos mediante red.
- Integración con otros sistemas de monitoreo.

La arquitectura propuesta permite incorporar estas funciones después de la etapa de adquisición sin alterar el principio fundamental del sistema.

16. Resumen de arquitectura

El sistema implementado sigue una arquitectura de adquisición y procesamiento de datos en la que el hardware OpenBCI captura y digitaliza las señales EEG, mientras que la Raspberry Pi ejecuta el software encargado de recibir, interpretar, organizar y almacenar los datos.

La estructura general es:



Esta arquitectura constituye la base del sistema de adquisición EEG de 16 canales desarrollado en el proyecto.